

Fiona McGinnis
Rodolfo Magana Lopez
Khiem Vu
Andres Benitez

Lab 5: Direct Mapped Cache

Assignment

- *Added Direct Mapped Cache L1 Instruction Memory to pipelined OTTER*
- *Ran testall.s on new OTTER with cache*
- *Compared testall.s performance to Lab 4*

Group Member Name	Contributions
Fiona McGinnis	Digital Design, Report
Rodolfo Magana Lopez	Digital Design, Debugging, Code Testing
Khiem Vu	Digital Design, Debugging, Code Testing, Report
Andres Benitez	Code Testing

Overview of Design

- ***Tasks***

Given the three modules (FSM, Cache, IMEM) on the lab assignment description, we were supposed to implement them into our pipelined OTTER. The goal was to successfully run testall.s using the direct mapped cache.
- ***Implementations for this lab***

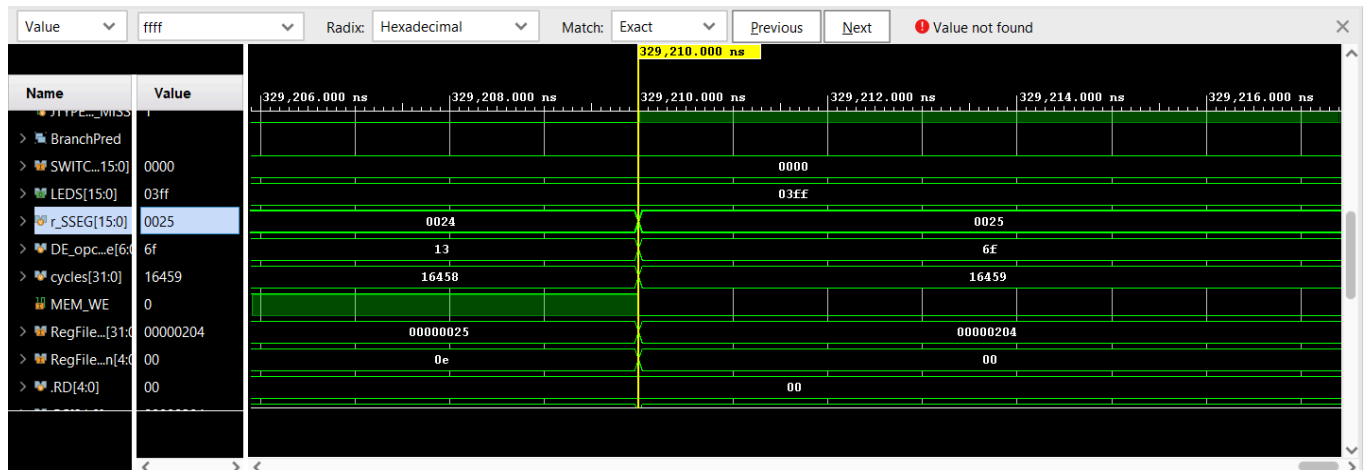
The IMEM from Lab 4 was removed, and the new IMEM from the lab description was added. We had added the block base address to make sure it aligned with our cache. Specifically, our cache consisted of 16 blocks and 8 words per block. The FSM, from the lab description, helped to send signals to stall the PC and update cache if there was a miss. Inside our actual pipeline code, we needed to add some additional signals and change some of the pipeline register logic to take into account cache misses, especially J-Type misses. In particular, the two most notable cache miss cases were J-Type cache misses and conditional branches. The working pipeline code has special J-Type miss logic that allows the first iteration of a J-Type cache miss to pass through the pipeline so that linking can occur, while stalling the PC, the fetch-decode register, and then making

sure the J-Type instruction that caused the cache miss doesn't execute a second time. For conditional branches, the logic wasn't as complicated since we didn't require linking, but the flush signal arising from a misprediction had to take precedence over the others (in fetch-decode register). The PC_WR signal also needed to be modified so that it knows when to write the PC when there is a cache miss.

Challenges Encountered

- The main challenge with this cache implementation was handling cache misses for instructions that redirect the PC.
- There was special JAL/JALR redirect cache miss logic added to make sure that the JAL/JALR redirect executed correctly when there was a cache miss. This involved making the PC_WR signal more exact and adding other signals to indicate J-Type cache misses and stalling correctly (see code comments for details).
- This special JAL/JALR redirect cache miss logic was unconditional, which we didn't take into account. The behavior of these cache misses were accidentally applied to conditional branches as well, which shouldn't be the case since we don't need to do linking. A mispredicted branch cache miss should behave normally similar to other instructions, but with a flush instead. Therefore, there needed to be some modification of the special J-Type cache miss logic along with changes in precedence of certain conditions in the pipeline registers.

Testall



Time: 329,210 ns

Cycles: 16,459

TESTALL PERFORMANCE

PIPELINE W/ HAZARD HANDLING (LAB 4)

Cycles	13,799
Runtime (ns)	276,010

CACHED PIPELINE (LAB 5)

Cycles	16,459
Runtime (ns)	329,210

The cached pipeline was approximately 19% slower than the normal pipelined OTTER.

TESTALL ON BASYS3 LINK

[OTTER Pipeline IMEM Cache Testall \(BASYS3\)](#)